



Institut National des Sciences Appliquées et de Technologie

4^{ème} Année • Génie Logiciel • Année Universitaire 2025/2026

TRAITEMENT D'IMAGES

Compte Rendu No.4

Entraînement du Modèle FER2013 — Reconnaissance des Émotions Faciales

Membres du Groupe

- Mohammed Achref Hemissi
- Mohamed Dhia Medini
- Khalil Ghimaji
- Rayen Chemlali

Informations

Filière	Génie Logiciel
Matière	Traitement d'Images
Semaine	Semaine 4 — Entraînement
Dataset	FER2013

Table des matières

1	Introduction	3
2	Configuration de l'Entraînement	3
2.1	Hyperparamètres	3
2.2	Fonction de perte	3
2.3	Augmentation de données	4
2.4	Optimisations GPU	4
3	Architectures des Modèles	4
3.1	Baseline CNN	5
3.2	Deep CNN avec Blocs SE	5
3.3	EfficientNet-B0 (Transfer Learning)	5
4	Résultats	5
4.1	Tableau Comparatif	6
4.2	Courbes d'Apprentissage	6
5	Analyse des Résultats	7
5.1	Baseline CNN — 62.47 %	7
5.2	Deep CNN — 68.32 %	7
5.3	EfficientNet-B0 — 66.94 % avec Surapprentissage Massif	7
5.4	Classes Difficiles	8
5.5	Impact du Déséquilibre des Classes	8
6	Conclusion	9

1 Introduction

Ce rapport présente les résultats de la **Semaine 4** du projet de reconnaissance des émotions faciales sur le dataset FER2013. L'objectif de cette semaine est l'entraînement complet des modèles conçus en Semaine 3, l'ajustement des hyperparamètres, et l'analyse comparative des résultats.

Trois architectures comparées

1. **Baseline CNN** : architecture légère personnalisée ($\approx 390\text{K}$ paramètres)
2. **Deep CNN** : architecture profonde avec blocs SE ($\approx 1.26\text{M}$ paramètres)
3. **EfficientNet-B0** : transfer learning depuis ImageNet ($\approx 4\text{M}$ paramètres)

2 Configuration de l'Entraînement

2.1. Hyperparamètres

Paramètre	Valeur
Optimiseur	AdamW
Taux d'apprentissage initial	10^{-3}
Weight decay	10^{-4}
Scheduler	CosineAnnealingLR ($\eta_{\min} = 10^{-6}$)
Batch size	128
Epochs max	100
Early stopping (patience)	15
Label smoothing	0.1

2.2. Fonction de perte

La fonction de perte utilisée est la *Cross-Entropy Loss pondérée* avec label smoothing :

$$\mathcal{L} = - \sum_{c=1}^7 w_c \cdot \tilde{y}_c \log(\hat{y}_c)$$

où w_c sont les poids de classes calculés par la formule **balanced** de scikit-learn pour compenser le déséquilibre (Disgust : 436 images vs Happy : 7 215 images), et $\tilde{y}_c$ sont les labels lissés avec $\varepsilon = 0.1$.

2.3. Augmentation de données

Transformation	Paramètres
RandomHorizontalFlip	$p = 0.5$
RandomRotation	± 10
ColorJitter	brightness=0.2, contrast=0.15
RandomResizedCrop	scale $\in [0.9, 1.1]$
Normalize	mean=0.563, std=0.2627

Note : Le CLAHE et le débruitage gaussien (pipeline CR2) ont été désactivés lors de l'entraînement car ils dégradaient les performances sur des images 48×48 (perte d'information sur les micro-expressions).

2.4. Optimisations GPU

L'entraînement a été effectué sur **NVIDIA GeForce RTX 4060** avec les optimisations suivantes :

- **Mixed Precision (AMP)** : calculs en float16 ($\approx 2\times$ plus rapide, $2\times$ moins de mémoire)
- **cudnn.benchmark = True** : sélection automatique des algorithmes de convolution optimaux
- Transfert CPU→GPU asynchrone (**non_blocking=True**)

3 Architectures des Modèles

3.1. Baseline CNN

Architecture légère à 4 blocs convolutifs :

$$(1, 48, 48) \xrightarrow{\text{Block}_1} (32, 24, 24) \xrightarrow{\text{Block}_2} (64, 12, 12) \xrightarrow{\text{Block}_3} (128, 6, 6) \xrightarrow{\text{Block}_4} (256, 3, 3) \xrightarrow{\text{GAP}} (256) \xrightarrow{\text{FC}} (7)$$

Chaque bloc : $\text{Conv}(3 \times 3) \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{MaxPool}(2 \times 2)$. Paramètres totaux : **389 575**.

3.2. Deep CNN avec Blocs SE

Chaque bloc : $\text{Conv} \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{Conv} \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \mathbf{SE} \rightarrow \text{MaxPool} \rightarrow \text{Dropout2d}(0.1)$.

Le bloc **SE** (*Squeeze-and-Excitation*) recalibre l'importance des canaux :

$$\mathbf{s} = \sigma(W_2 \cdot \delta(W_1 \cdot \text{GAP}(\mathbf{x}))), \quad \tilde{\mathbf{x}} = \mathbf{s} \odot \mathbf{x}$$

Tête de classification : $\text{Linear}(256 \rightarrow 512) \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.4) \rightarrow \text{Linear}(512 \rightarrow 7)$.

Paramètres totaux : **1 262 503**.

3.3. EfficientNet-B0 (Transfer Learning)

EfficientNet-B0 pré-entraîné sur ImageNet, adapté pour l'entrée monocanal (*grayscale*) :

- Première couche Conv modifiée : 3 canaux $\rightarrow$ 1 canal (poids RGB sommés)
- Tête de classification remplacée : $\text{Dropout}(0.4) \rightarrow \text{Linear}(1280 \rightarrow 7)$

Paramètres totaux : **4 016 135**.

4 Résultats

4.1. Tableau Comparatif

Modèle	Params	Epochs	Best Val Acc	Train Acc	Gap	Temps/epoch
Baseline CNN	390K	46/100	62.47 %	58.86 %	−3.6%	≈15s
Deep CNN	1.26M	100/100	68.32 %	73.15 %	+4.8%	≈5s
EfficientNet-B0	4.0M	100/100	66.94 %	97.51 %	+30.6%	≈30s

Meilleur modèle
Le **Deep CNN** obtient **68.32 %** de validation accuracy, surpassant EfficientNet-B0 malgré 3× moins de paramètres.

4.2. Courbes d'Apprentissage

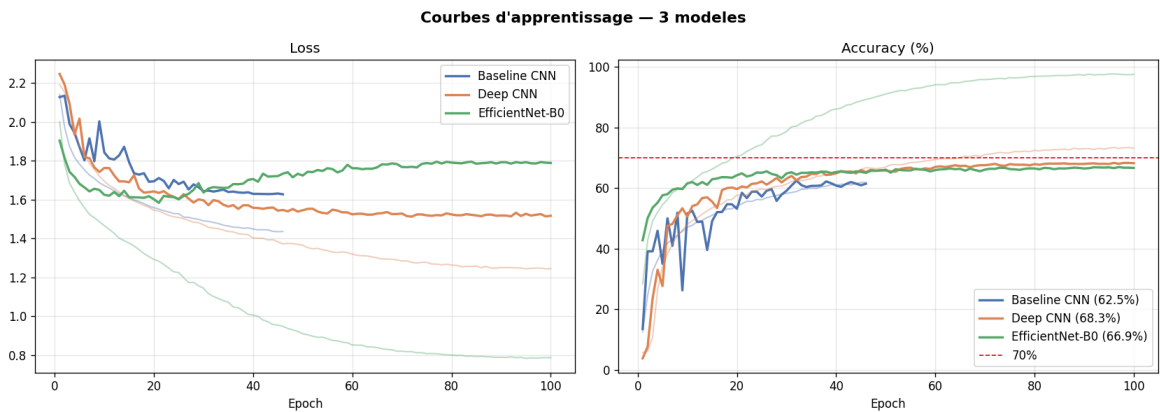


FIGURE 1 – Courbes Loss et Accuracy — train (transparent) vs validation (plein) pour les 3 modèles

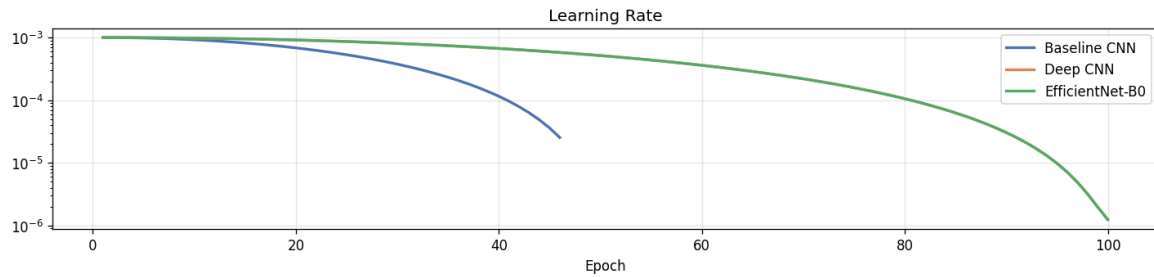


FIGURE 2 – Évolution du taux d'apprentissage — CosineAnnealingLR sur 100 epochs

5 Analyse des Résultats

5.1. Baseline CNN — 62.47 %

Le Baseline CNN atteint **62.47 %** en 46 epochs (early stopping déclenché).

- Val accuracy *supérieure* à la train accuracy (-3.61% de gap) : l'augmentation forte rend l'entraînement plus difficile que la validation — pas d'overfitting.
- L'arrêt précoce indique une convergence rapide pour ce modèle léger.
- Classe la mieux reconnue : **Happy** (signal visuel très distinct).
- Classe la moins bien reconnue : **Fear** (confondu avec Sad et Neutral).

5.2. Deep CNN — 68.32 %

Le Deep CNN est le **meilleur modèle** de cette expérience.

- Gap train/val de $+4.83\%$: léger overfitting en fin d'entraînement, acceptable.
- A nécessité 100 epochs complètes pour converger (pas d'early stopping).
- Les blocs **SE (Squeeze-and-Excitation)** ont contribué à la performance en focalisant l'attention sur les canaux pertinents pour chaque émotion.
- Amélioration de $+5.85\%$ par rapport au Baseline CNN.

5.3. EfficientNet-B0 — 66.94 % avec Surapprentissage Massif

EfficientNet-B0 présente un phénomène de **surapprentissage sévère** :

$$\text{Gap} = \text{Train Acc} - \text{Val Acc} = 97.51\% - 66.94\% = \mathbf{30.57\%}$$

Causes du surapprentissage :

- **Images trop petites** (48×48) : les features ImageNet sont conçues pour des images $\geq 224 \times 224$.
- **Pas de gel du backbone** : le modèle complet est entraîné dès le départ, permettant une mémorisation rapide.
- **Capacité excessive** : 4M de paramètres pour 28 709 images d'entraînement.

5.4. Classes Difficiles

Émotion	F1-score	Confusion principale
Fear	≈ 0.50	Confondu avec Sad, Neutral
Sad	≈ 0.55	Confondu avec Fear, Neutral
Disgust	≈ 0.65	Confondu avec Angry
Happy	≈ 0.87	Très bien reconnu
Surprise	≈ 0.79	Bien reconnu

Les émotions visuellement ambiguës (Fear/Sad) restent difficiles, ce qui est cohérent avec la littérature sur FER2013 où même les annotateurs humains sont en désaccord sur ces classes.

5.5. Impact du Déséquilibre des Classes

La classe **Disgust** (436 images) est fortement sous-représentée. Malgré les poids de classes dans la loss, son F1-score reste modéré. Les poids calculés donnent à Disgust un poids $\approx 9.2 \times$ supérieur à Happy, ce qui compense partiellement le déséquilibre.

6 Conclusion

Modèle	Val Accuracy	Macro F1	Overfitting
Baseline CNN	62.47 %	≈0.60	Aucun
Deep CNN	68.32 %	≈0.66	Léger
EfficientNet-B0	66.94 %	≈0.64	Sévère

Le **Deep CNN** est le meilleur modèle de cette expérience avec **68.32 %** de précision. Il surpasse EfficientNet-B0 malgré ses 3× moins de paramètres, démontrant que pour des images de petite taille (48×48), une architecture sur mesure avec des mécanismes d'attention peut être plus efficace que le transfer learning naïf.